

10a — Database Indexes

Key Concepts

Term	Definition
Index	Separate data structure maintaining pointers to rows for fast lookup
B-Tree	Default index type; balanced tree giving $O(\log n)$ lookups
Full Table Scan	Checking every row — happens when no index exists
Cardinality	Number of distinct values in a column; high cardinality = better index candidate
Covering Index	Index contains all columns a query needs — no table lookup required
Composite Index	Index on multiple columns; order matters
Partial Index	Index on a subset of rows (e.g. <code>WHERE status = 'active'</code>)
Replication Lag	Delay between primary write and replica catching up

How It Works

- 1. Query arrives: `WHERE email = 'anil@email.com'`
- 2. Without index → full table scan → checks every row → $O(n)$
- 3. With B-Tree index → traverses tree → reaches matching leaf → $O(\log n)$
- 4. DB returns result directly from index pointer

Composite index left-to-right rule:

- Index on `(country, city)` helps `WHERE country = ?` ☒
- Index on `(country, city)` does NOT help `WHERE city = ?` alone ☒

Function kills index:

- `WHERE YEAR(created_at) = 2024` → index on `created_at` not used ☒
 - `WHERE created_at >= '2024-01-01'` → index used ☒
-

Types of Indexes

Type	Best For
B-Tree (default)	Range queries, equality, sorting
Hash	Exact equality only
Composite	Multiple column filters
Partial	Subset of rows

Type	Best For
Full-Text	Text search (LIKE '%word%')
Covering	All needed columns inside the index

Trade-offs

- **Reads:** Faster — $O(\log n)$ vs $O(n)$
- **Writes:** Slower — every INSERT/UPDATE/DELETE must update all indexes
- **Storage:** Indexes take disk space (can match table size)
- **Rule:** Index columns used in WHERE, ORDER BY, JOIN — not every column

Industry Examples

- **Twitter** — index on (`user_id`, `created_at`) for fast timeline queries
- **Amazon** — composite index on (`category`, `price`) for filtered product search
- **LinkedIn** — partial index on `status = 'open'` for job listings
- **Uber** — index on (`driver_id`, `trip_status`) to find active rides instantly

Common Mistakes

Mistake	Problem
Over-indexing	Write performance degrades
Wrong composite column order	Index unused for partial filters
Wrapping column in function	Index not used
Missing foreign key index	Slow JOINS
Low cardinality standalone index	DB may skip index entirely

Interview Questions

Q: What is a database index? A: A separate data structure (usually B-Tree) that allows the DB to find rows without a full table scan — trades write speed and storage for read speed.

Q: Why not index every column? A: Every write must update all indexes — too many indexes slows inserts, updates, and deletes significantly.

Q: What is a covering index? A: An index containing all columns a query needs, so the DB never reads the actual table — fastest possible read.

Q: Why does column order matter in composite indexes? A: The index is sorted by the first column first. Filtering only on the second column can't use the index efficiently.

Q: When would you use a partial index? A: When only a subset of rows is queried frequently — e.g. index only active users, not deleted ones — keeps the index small and fast.